Security Best Practices for Generative AI in the Enterprise

Dell AI Platforms

Abstract

This white paper explores the key considerations for safeguarding AI infrastructure, including the Dell AI Platforms, offering insights and best practices to help organizations ensure their AI environments remain secure, scalable, and effective.

Dell Technologies AI Solutions

Part Number: H04511.1
October 2025

DELL Technologies

Notes, cautions, and warnings

NOTE: A NOTE indicates important information that helps you make better use of your product.

CAUTION: A CAUTION indicates either potential damage to hardware or loss of data and tells you how to avoid the problem.

WARNING: A WARNING indicates a potential for property damage, personal injury, or death.

Contents

1

Topics:

Introduction

Overview

Artificial intelligence is transforming the way organizations operate, innovate, and compete. By unlocking the potential of vast datasets, AI enables businesses to gain valuable insights, drive automation, and tackle complex challenges at unprecedented speeds. However, as organizations rush to adopt AI, it is crucial to recognize that this powerful technology also introduces new security vulnerabilities and risks. From securing large-scale datasets to protecting AI models against adversarial threats, the need for robust and resilient security infrastructure has never been more critical.

The Dell AI Factory is Dell's approach to accelerate AI innovation in organizations of all sizes. It consists of a portfolio of products, solutions, and services that are optimized for AI workloads. It brings together these AI-optimized technologies with an open ecosystem of partners, validated and integrated solutions, expert services, and best practices to help customers achieve AI outcomes faster. There are several Dell AI Platforms that provide predesigned, scalable, and validated architectures for AI infrastructure in the Dell AI Factory portfolio.

The Dell AI Platforms exist at the intersection of rapidly evolving AI technology and the ever-challenging security landscape. This unique position brings both incredible opportunities and significant risks. This document describes some of the potential threats to an AI platform and offers guidance about how to mitigate them effectively.

The security of any system exists in relation to the system's threat profile: the value of a system's assets, the likelihood of an attempted attack, and the nature of potential attackers in terms of their resources, capabilities, and incentives. Furthermore, there is often a tension between security and convenience: extra security measures may interfere with desirable use cases or lead to false-positive results. Therefore, Dell Technologies advises customers and architects to consider their specific operational contexts and conduct appropriate analysis to validate the applicability of the recommendations in this guide. Depending on the threat profile, additional services may be advisable.

The following sections detail some common security configuration procedures, followed by guidance related to categories of potential threats and their respective mitigations.

Document purpose

This white paper explores the key considerations for safeguarding AI infrastructure, including the Dell AI Platforms. It offers insights and best practices to help organizations ensure that their AI environments remain secure, scalable, and effective.

Always deploy and use the hardware and software that is recommended in the Dell AI Platform designs in a secure manner. Be aware of the potential threats and mitigation strategies that are described in this document.

Audience

This document is intended for practitioners that are interested in the implementation of AI infrastructure platforms and solutions for generative AI, including professionals and stakeholders involved in the development, deployment, and management of generative AI systems, with a focus on security. Some knowledge of AI development and life cycle, generative AI principles, and terminology is assumed.

Revision history

The following table lists the revision history.

Table 1. Revision history

Date	Version	Change summary
October 2025	1.1	Added application and DAP security considerations.
May 2025	1.0	Initial release.

General security considerations

Security of a system as a whole begins with the security of its components and subsystems. Configure each of your Dell products based on the guidance provided in the individual security configuration guides that are listed in the References section of this paper.

For additional information about security, compliance, and privacy, go to the Dell Security and Trust Center for additional information about security, compliance, and privacy.

Integrating internal PKI

The public key infrastructure (PKI) is a system for managing digital certificates and public-key encryption. A Certificate Authority (CA) is an entity that uses public-key (asymmetric) encryption to enable trust over the Internet. Most operating systems, web browsers, and other software are shipped with the public keys of many CAs. Users can use them to establish trust with entities that have a relationship with one of the CAs. This technology underpins HTTPS, code signing, and more.

Enterprise IT environments often host their own PKIs and issue private CA Certificates for many reasons, including securing internal services and deep packet inspection (DPI) at the network perimeter. DPI technologies operate in between the client and the server. The secure connection to the server is terminated so that the contents can be inspected, the connection to the client is secured with certificates that the internal CA signs. In either case, an internal service or an external service subject to DPI, software such as operating systems and browsers, does not trust the internal CA by default. Therefore, if an AI Platform is in an environment where, for example, essential software sources are subject to DPI, then users must establish trust of the internal CA, rather than use insecure connections.

AI Platform administrators can distribute internal CA certificates within the Kubernetes cluster by creating a ConfigMap to store the CA certificate. For more information, see the Kubernetes documentation.

To configure an operating system to trust an internal Certificate Authority, customers can install a CA certificate in the trust store. See the Ubuntu documentation for more information.

Using registry mirrors

A registry mirror is an alternative instance of a container image registry. For example, customers can use an external registry mirror to access a registry from behind a regional firewall. Often, customers host an internal registry mirror that acts as a caching pass-through to the public registry. This method results in faster downloads, reduced attack surface, and avoidance of rate limit restrictions.

The AI Platform's Kubernetes Container Runtime, containerd, is responsible for pulling images, so it must be configured to use the registry mirror. For guidance, see the containerd documentation: Configure Image Registry.

If the registry mirror requires credentials or is secured with an internal Certificate Authority (detailed in the previous section), then containerd must also be configured. For guidance, see Registry Configuration - Introduction.

Security considerations for AI applications

AI applications are typically implemented using multiple components and subsystems. These applications often use a modular architecture where alternatives are possible for supporting components such as vector databases, key-value stores, and document stores. AI applications and their components are often integrated with authentication and authorization systems, PKI systems, external data sources, and storage systems. When analyzing AI applications, consider all components and subsystems that are deployed or used.

AI applications deployed with Dell Automation Platform blueprints

Dell Automation Platform (DAP) blueprints automate the deployment of complete AI applications. As part of the deployment process, a blueprint may deploy supporting components or use additional blueprints to satisfy prerequisites. AI applications may also involve additional configuration and setup through application user interfaces after deployment with a blueprint. The documentation for individual blueprints is available from the DAP catalog. The documentation includes details of the blueprint deployment options and components deployed. When deploying AI applications using blueprints from the DAP catalog, all security considerations and guidance in this document remain relevant.

Dell Automation Platform security configuration

For information about security features, capabilities, and maximizing security of the Dell Automation Platform installation, see the the Dell Automation Platform Security Configuration Guide.

AI threats

AI model theft, reconstruction, and exfiltration

Adversaries may extract a functional copy of a private model by repeatedly querying the ML Model Inference API Access of the victim. The adversary can collect the target model inferences into a dataset that are then used as labels for training a separate model offline that mimics the behavior and performance of the target model. This attack also occurs when proprietary models, being valuable intellectual property, are compromised, physically stolen, copied, or when weights and parameters are extracted to create a functional equivalent.

Mitigation strategies include:

- **Secure access controls**
  - Implement secure access controls for the ML Model Inference API.
  - Use secure authentication and authorization mechanisms for the model storage and inference systems, including the Dell PowerScale OneFS server.
- **Data encryption and storage**
  - Implement data encryption and secure data storage.
  - Use a secure method to store and manage user credentials.
- **Regular updates and patching**
  - Regularly review and update the firmware and software of the PowerEdge servers.
  - Regularly update and patch PowerScale OneFS server software.
  - Use a vulnerability scanner to identify and address vulnerabilities in the software.
- **Additional security measures**
  - Implement model watermarking and model obfuscation to protect intellectual property.
  - Implement model obfuscation and encryption to prevent model extraction.
- **Network segmentation**
  - Implement network segmentation to limit exposure of insecure interfaces.

    - Use a secure method to access the PowerScale OneFS server.
- **Monitoring and logging**
    - Monitor system activity to detect and respond to potential security threats.
    - Implement secure logging to detect and respond to potential security threats.
- **User education**

  Educate users about the importance of secure ML Model Inference API and model training.
- **Secure model training techniques**
    - Use secure model training techniques to prevent adversaries from re-creating the underlying model.
    - Implement access controls to prevent adversaries from accessing the ML Model Inference API.

AI data poisoning

Adversaries can contaminate the training data of an ML system to achieve a wanted outcome at inference time. With influence over training data, an attacker can create backdoors so that an arbitrary input results in a particular output. Adversaries might introduce malicious data providers, creating poisoned training data and publishing it to a public location, tricking victims into using it. The poisoned dataset might be a novel dataset or a poisoned variant of an existing open-source dataset.

Types of data poisoning include:

- **Training data poisoning**—Injecting malicious data during the training phase to manipulate model behavior
- **Inference data poisoning**—Manipulating input data during the inference phase to deceive the model
- **Poisoning pretrained models**—Introducing malicious data into pretrained models with baked-in data, algorithms, and third-party and external libraries

Mitigation strategies include:

- **Secure access controls**
    - Implement secure access controls for the ML system.
    - Use secure authentication and authorization mechanisms.
- **Data Validation and sanitization**
    - Implement data validation and sanitization techniques to prevent malicious data from being used in the training process.
    - Secure the training data to prevent adversaries from contaminating it.
- **Secure ML model development practices**
    - Use secure data sources and monitor data use to detect potential security incidents.
    - Implement secure model training techniques to prevent adversaries from re-creating the underlying model.
- **Regular updates and patching**
    - Regularly update and patch the PowerScale OneFS server software.
    - Use a vulnerability scanner to identify and address vulnerabilities in the software.
- **Network segmentation**
    - Implement network segmentation to limit exposure of insecure interfaces.
    - Use a secure way to access the PowerScale OneFS server.
- **Monitoring and logging**
    - Monitor system activity to detect and respond to potential security threats.
    - Implement secure logging to detect and respond to potential security threats.
- **User education**

  Educate users about the importance of secure training data and data validation.
- **Additional security measures**
    - Implement model obfuscation and encryption to prevent model extraction.
    - Use secure ways to manage and control access to training data.

AI/ML supply chain compromise

Hardware or software that is used in AI systems might be compromised long before deployment.

Adversaries might gain initial access to a system by compromising unique portions of the ML supply chain, which could include GPU hardware, data and its annotations, parts of the ML software stack, or the model itself.

Mitigation strategies include:

- **Run containers and pods with least privileges**

  Ensure that containers and pods operate with the minimum privileges that are necessary to reduce the potential impact of a compromise.
- **Network separation**

  Use network segmentation to control the extent of damage a compromise can cause. This method limits the movement of adversaries within the network.
- **Firewalls**

  Implement firewalls to restrict unnecessary network connectivity that reduces exposure to potential threats.
- **Secure hardware**

  Use hardware that has been validated for security to prevent hardware-based attacks.
- **Protect data and annotations**

  Secure data and its annotations to prevent tampering and unauthorized access.
- **Secure ML software components**

  Use trusted and secure ML software components to minimize vulnerabilities.
- **Model validation**

  Validate that ML models to ensure that they have not been tampered with before deployment.
- **Secure development practices**

  Implement secure development practices for ML software to reduce the risk of introducing vulnerabilities during development.
- **Access controls and authentication**

  Employ robust access controls and authentication mechanisms for sensitive ML-related data and resources.
- **Secure protocols for data transfer**

  Use secure protocols for data transmission and storage to protect data integrity and confidentiality.
- **Regular updates and patching**

  Regularly update and patch ML software to address known vulnerabilities.
- **Monitoring for suspicious activity**

  Continuously monitor systems for signs of suspicious activity to detect and respond to potential threats promptly.

AI reinforcement learning rewards hacking

Reward systems in AI reinforcement learning environments can be manipulated or malfunction, leading to inappropriate rewards.

Mitigation strategies include:

- **Robust reward system design and testing**

  Design and thoroughly test reward systems to prevent manipulation. Ensure that rewards are meaningful and consistent through techniques like reward shaping and normalization.
- **Monitoring for Anomalies**

  Continuously monitor reward systems for anomalies to ensure that they align with wanted behaviors.
- **Secure Reinforcement Learning Algorithms**

  Use secure reinforcement learning algorithms to minimize vulnerabilities.
- **Intrinsic Motivation and Exploration-Exploitation Trade-Off**

  Implement techniques such as intrinsic motivation and exploration-exploitation trade-offs to prevent reward hacking.
- **Validation and Verification**

  Validate rewards through rigorous testing and verification processes.
- **Secure Protocols and Access Controls**

  Use secure protocols for data transfer and storage, and implement robust access controls and authentication mechanisms.

AI model inversion

An attacker recovers the features that are used to train the model.

Model inversion attacks involve adversaries recovering the features that are used to train a model, potentially compromising private data.

Mitigation strategies include:

- **Homomorphic Encryption and Federated Learning**

  Use homomorphic encryption or federated learning to protect model features. These techniques ensure that data remains encrypted during processing and training.
- **Encrypted Feature Storage**

  Store features in encrypted form to prevent unauthorized access.
- **Access Controls**

  Limit access to sensitive data and features to authorized personnel only.
- **Differential Privacy**

  Implement differential privacy techniques to obscure sensitive information and protect individual data points.
- **Regular Audits** Regularly audit access controls and feature storage to ensure compliance with security policies.

  Implement differential privacy techniques to obscure sensitive information and protect individual data points.
- **Secure Feature Extraction**

  Use secure methods for feature extraction to minimize the risk of data leakage.
- **Data Anonymization**

  Anonymize data to protect private information during model training.
- **Secure Multi-Party Computation**

  Implement secure multiparty computation to protect data during collaborative training processes.
- **Feature Hashing and Dimensionality Reduction**

  Use feature hashing or dimensionality reduction to reduce the impact of recovered features.
- **Secure Model Training Protocols**

  Apply secure protocols for model training to ensure data integrity and confidentiality.

Creating a back door in an AI system

Adversaries can introduce a backdoor into an AI system by poisoning training data, interfering with the training process, or injecting a payload into the model file.

Mitigation strategies include:

- **Data Validation and Sanitization**

  Implement robust data validation and sanitization techniques to prevent the inclusion of poisoned training data.
- **Secure and Reproducible Training Processes**

  Use secure and reproducible training processes to ensure the integrity of the training environment.
- **Monitoring for Anomalies**

  Continuously monitor the training process for anomalies to detect and prevent tampering.
- **Model Explainability and Interpretability**

  Employ techniques such as model explainability and interpretability to identify and prevent back doors.
- **Model Integrity Checks**

  Implement model integrity checks to detect and prevent backdoor attacks.
- **Secure and Trusted Data Sources**

  Use secure and trusted data sources for training to minimize the risk of data poisoning.
- **Robust and Repeatable Training Processes**

  Ensure that training processes are robust and repeatable to maintain consistency and security.
- **Secure Model Development and Deployment Practices**

  Implement secure practices for model development and deployment to protect against backdoor attacks.

- **Model Monitoring and Anomaly Detection**

  Use model monitoring and anomaly detection techniques to identify potential back doors.
- **Digital Signatures and Verification**

  Use digital signatures and verification to ensure the integrity of model files.

AI perturbation attack on training data or models

Carefully crafted input can be used to confuse and deceive the model.

Adversarial data are inputs to a machine learning model that have been modified to cause a wanted effect in the target model for the adversary, potentially causing misclassification, missed detections, or maximizing energy consumption.

Mitigation strategies include:

- **Adversarial Training**

  Implement adversarial training techniques to improve the model's robustness against adversarial inputs.
- **Data Augmentation**

  Use data augmentation to increase the model's generalizability and resilience to manipulated inputs.
- **Input Validation and Sanitization**

  Validate and sanitize data to prevent the inclusion of poisoned or manipulated inputs.
- **Early Stopping and Model Pruning**

  Use techniques such as early stopping and model pruning to prevent overfitting, which can make models more susceptible to adversarial attacks.
- **Regular Audits and Updates**

  Regularly audit and update the model to ensure that it remains resilient against new adversarial techniques.
- **Model Robustness Checks**

  Employ model robustness checks to detect and prevent adversarial attacks.
- **Model Monitoring and Anomaly Detection**

  Implement model monitoring and anomaly detection to identify potential adversarial data.
- **Data Preprocessing and Normalization**

  Apply data preprocessing and normalization to reduce the impact of adversarial inputs.
- **Intrusion Detection Systems**

  Use intrusion detection systems to identify and block adversarial attacks.

Alteration of information consumed by AI systems

Attackers can introduce noise or variations to the information consumed by AI systems, potentially causing misclassification, missed detections, or maximizing energy consumption.

Mitigation strategies include:

- **Data Validation and Sanitization**

  Implement robust data validation and sanitization techniques to prevent the inclusion of poisoned or manipulated data.
- **Robustness Techniques**

  Use robustness techniques such as data augmentation and adversarial training to increase the model's generalizability and resilience to manipulated inputs.
- **Monitoring for Anomalies**

  Continuously monitor model performance for anomalies to detect and prevent tampering.
- **Regular Audits and Updates**

  Regularly audit and update the model to ensure that it remains resilient against new noise or variation attacks.
- **Data Quality Checks**

  Implement data quality checks to ensure the integrity of the data being consumed by the AI system.
- **Data Normalization and Denoising**

  Use data normalization and denoising techniques to reduce the impact of noise or variations.

- **Model Robustness Checks**

  Employ model robustness checks to detect and prevent data poisoning attacks.
- **Input Validation and Data Processing**

  Implement robust input validation and data processing mechanisms, such as noise reduction techniques or data normalization.
- **Model Monitoring and Anomaly Detection**

  Use model monitoring and anomaly detection techniques to identify potential attacks.
- **Noise Reduction and Data Cleansing**

  Apply noise reduction and data cleansing techniques to mitigate the effects of noise or variation.

Denial of service in an AI system

A threat actor can cause a denial of service (DoS) condition in an AI system by inputting queries that result in excessive resource consumption or lengthy computation times, thereby limiting access or use.

Mitigation strategies include:

- **Load Balancing and Resource Allocation**

  Implement load balancing and resource allocation techniques to distribute workloads evenly and prevent DoS attacks.
- **Optimize Model Performance**

  Optimize model performance and reduce computation time to minimize the attack surface.
- **Model Pruning and Compression**

  Use techniques such as model pruning and compression to reduce the computational overhead and attack surface.
- **Monitoring for Anomalies**

  Continuously monitor resource consumption and model performance for anomalies to detect and prevent tampering.
- **Regular Audits and Updates**

  Regularly audit and update the model to ensure that it remains resilient against new DoS techniques.
- **Query Rate Limiting**

  Implement query rate limiting to control the number of queries that are processed and prevent resource exhaustion.
- **Resource Monitoring and Control**

  Use resource monitoring and control mechanisms to manage resource utilization effectively.
- **Request Throttling**

  Implement request throttling to limit the rate at which queries are processed.
- **Query Validation and Optimization**

  Validate and optimize queries to ensure that they do not consume excessive resources.
- **Caching**

  Use caching techniques to store frequently accessed data and reduce the load on the system.
- **Resource Allocation Controls**

  Apply resource allocation controls to manage the distribution of resources effectively.

AI transfer learning attack

Transfer learning attacks occur when an attacker trains a model on one task and then fine-tunes it on another task. This act causes the model to behave in an undesirable way, potentially tricking an AI system into using crafted data or models.

Mitigation strategies include:

- **Model Validation and Sanitization**

  Implement robust model validation and sanitization techniques to prevent transfer learning attacks.
- **Fine-Grained Feature Extraction and Adversarial Training**

  Use techniques such as fine-grained feature extraction and adversarial training to increase model robustness.
- **Monitoring for Anomalies**

  Continuously monitor model performance for anomalies to detect and prevent tampering.

- **Regular Audits and Updates**

  Regularly audit and update the model to ensure that it remains resilient against new transfer learning attacks.
- **Secure Fine-Tuning Techniques**

  Use secure fine-tuning techniques to maintain model integrity.
- **Model Explainability and Interpretability**

  Employ techniques such as model explainability and interpretability to identify and prevent transfer learning attacks.
- **Robust Model Validation and Testing**

  Implement robust model validation and testing processes, including adversarial testing and transfer learning attack simulations.
- **Model Monitoring and Anomaly Detection**

  Use model monitoring and anomaly detection techniques to identify potential transfer learning attacks.
- **Secure Model Updating Protocols**

  Apply secure protocols for model updating to ensure data integrity and confidentiality.
- **Transfer Learning Detection Techniques**

  Implement transfer learning detection techniques to monitor and control model behavior.

AI system is susceptible to prompt injection

User queries can cause unintended or unsafe behavior.

Carefully crafted manipulated prompts can cause unintended behavior, information disclosure, system exploits, and other issues in an AI system.

Mitigation strategies include:

- **Input Validation and Sanitization**

  Implement robust input validation and sanitization techniques to prevent prompt manipulation attacks.
- **Natural Language Processing (NLP) and Machine Learning-Based Approaches**

  Use NLP and machine learning-based approaches to detect and prevent manipulated prompts.
- **Monitoring for Anomalies**

  Continuously monitor model performance for anomalies to detect and prevent tampering.
- **Regular Audits and Updates**

  Regularly audit and update the model to ensure that it remains resilient against new prompt manipulation techniques.
- **Secure Prompt Engineering**

  Use secure prompt engineering techniques to maintain the integrity of the AI system.
- **Prompt Filtering, Ranking, and Response Validation**

  Implement techniques such as prompt filtering, prompt ranking, and response validation to identify and prevent potential attacks.
- **Model Robustness Checks**

  Employ model robustness checks to detect and prevent manipulated prompt attacks.
- **Input Normalization and Encoding**

  Use input normalization and encoding techniques to prevent unintended behavior.
- **Prompt Analysis and Modeling**

  Apply prompt analysis and modeling techniques to ensure the security of the input processing.

AI system is granted excessive agency

An AI system granted privileges or capabilities beyond what is necessary or safe can lead to unintended consequences. This scenario can occur due to excessive permissions, functionality, or autonomy of Large Language Model (LLM)-based systems.

Mitigation strategies include:

- **Role-Based Access Control (RBAC)**

Implement RBAC to restrict the privileges and capabilities of AI systems. Regularly review and update permissions and access controls to ensure that they are secure and aligned with business needs.

- **Least Privilege Principle**

  Use the least privilege principle to limit the privileges and capabilities of AI systems to only what is necessary for them to perform their intended functions.
- **Monitoring and Logging**

  Implement monitoring and logging to detect potential security incidents and anomalies.
- **Secure Development Practices**

  Use secure development practices to ensure that AI systems are designed and developed with security in mind.
- **Configuration Management**

  Configure AI platform and cluster management systems such as Dell Omnia to grant only the necessary privileges and capabilities to AI and automation systems. Implement secure access control and auditing mechanisms to detect and prevent unauthorized access or actions.
- **Web Application Firewall (WAF)**

  Consider using a WAF to detect and prevent privilege escalation attacks.
- **Auditing and Monitoring**

  Ensure that all AI and automation systems are properly audited and monitored.
- **Limit Functionality and Autonomy**

  Limit the functionality and autonomy of LLM-based systems to prevent excessive agency.
- **Secure Coding Practices**

  Follow secure coding practices to prevent unintended consequences and ensure that AI system access is properly configured.
- **Regular Reviews and Updates**

  Regularly review and update system capabilities to ensure that they remain secure.

Insecure retrieval augmented generation system

Retrieval Augmented Generation (RAG) systems can be misconfigured or contain security vulnerabilities, potentially leading to issues such as information disclosure, denial of service, spoofing, and tampering.

Mitigation strategies include:

- **Secure Configuration and Monitoring**

  Implement secure configuration and monitoring of RAG systems to prevent misconfiguration and security vulnerabilities.
- **Regular Updates and Patching**

  Regularly update and patch RAG systems to address known vulnerabilities.
- **Secure Development Practices**

  Use secure development practices to ensure that RAG systems are designed and developed with security in mind.
- **Monitoring and Logging**

  Implement monitoring and logging to detect potential security incidents and anomalies.
- **Secure APIs and Interfaces**

  Use secure APIs and interfaces for interacting with RAG systems.
- **Configuration Management**

  Configure systems such as Omnia to properly secure RAG systems, including implementing secure data storage and access controls, as well as regular security audits and updates.
- **Web Application Firewall (WAF)**

  Consider using a WAF to detect and prevent security vulnerabilities and misconfigurations.
- **Input Validation and Sanitization**

  Ensure proper input validation and sanitization to prevent security vulnerabilities.
- **Secure Protocols and Access Controls**.

  Use secure protocols and implement access controls to manage system capabilities effectively.
- **Regular Reviews and Updates**

Regularly review and update system capabilities to ensure that they remain secure.

- **Log Data Analysis**

  Regularly review and analyze log data to identify potential security threats.
- **Secure Coding Practices**

  Follow secure coding practices to prevent security vulnerabilities and unintended consequences.

AI system reveals sensitive information

An AI system revealing sensitive information can lead to privacy violations and unauthorized data access. This scenario can occur through various stages of the AI life cycle, including data collection, training, and deployment.

Mitigation strategies include:

- **Data Encryption and Access Controls**

  Implement data encryption and access controls to protect sensitive information.
- **Regular Reviews and Updates**

  Regularly review and update AI system configurations to ensure that they are secure and aligned with business needs.
- **Secure APIs and Interfaces**

  Use secure APIs and interfaces for data access and manipulation.
- **Monitoring and Logging**

  Implement monitoring and logging to detect potential security incidents and anomalies.
- **Secure Development Practices**

  Use secure development practices to ensure that AI systems are designed and developed with security in mind.
- **Configuration Management**

  Configure systems such as Omnia to properly secure AI systems, including implementing secure data storage and access controls, as well as regular security audits and updates.
- **Data Anonymization and Encryption**

  Consider using data anonymization and encryption techniques to protect sensitive information.
- **Secure Data Collection and Storage**

  Implement secure data collection and storage practices, ensuring adequate data anonymization and pseudonymization.
- **Secure Communication Protocols**

  Use secure communication protocols for data transmission.
- **Data Protection Policies**

  Implement secure data protection policies and use secure data storage mechanisms.
- **Access Restrictions**

  Restrict access to sensitive data and ensure that AI system access is properly configured.
- **Secure Coding Practices**

  Follow secure coding practices to prevent sensitive information revelation and ensure that AI system usage is properly monitored.

Software threats

Malicious actions on Kubernetes cluster

An attacker can gain access to a Kubernetes cluster to perform actions such as data theft or service manipulation.

Attackers might try to compromise Kubernetes clusters for various reasons. These reasons include espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for activities such as cryptocurrency mining. Attackers can obtain initial access through several means:

- Using compromised credentials

- Compromising container images in a registry
- Obtaining the kubeconfig file
- Exploiting vulnerabilities in the container (base image and any included binaries)
- Exposure of insecure interfaces which may reveal sensitive information
- Accessing the Kubernetes Dashboard

Mitigation strategies include:

- **Strong Password Policies**

  Implement strong password policies to secure credentials.
- **Secure Container Images**

  Use secure container images and regularly update and patch them. Validate container images using a vulnerability scanner.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms such as OAuth or LDAP to prevent the use of compromised credentials.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Regular Updates and Patches**

  Regularly update and patch container images in the registry.

Running arbitrary code inside a cluster

Attackers with access and sufficient privileges can run code of their choice.

Attackers might try to compromise Kubernetes clusters for various reasons, such as hijacking resources for cryptocurrency mining. They can run code inside a cluster through several methods:

- Using kubectl exec
- Creating and injecting a new container that they control
- Injecting a sidecar container
- Remote code execution or other vulnerabilities that allow code injection
- Using SSH in a container (using stolen credentials or through credential stuffing)

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can run code within the cluster.
- **Restrict kubectl exec**

  Limit the use of kubectl exec to admin users only.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure Sidecar Containers**

  Use secure methods to inject sidecar containers.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent remote code execution.
- **Secure SSH Connections**

  Secure SSH connections within containers and use secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

Implement network segmentation to limit the exposure of insecure interfaces.

- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.

Attacker maintains persistence on a cluster

If initial access is lost, attackers can maintain access or have a means of reestablishing access.

Attackers might try to compromise Kubernetes clusters for various reasons, such as performing espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for cryptocurrency mining. They maintain persistence on a cluster through several methods:

- Using features such as DaemonSets or Deployments to make attacker-controlled containers persistent
- Using container-to-host hostPath mounts
- Running attacker-controlled code through Kubernetes CronJob jobs
- Setting an admissions controller webhook to inject backdoors and other malicious code during regular cluster operations

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can deploy and manage resources within the cluster.
- **Restrict DaemonSets and Deployments**

  Limit the use of DaemonSets and Deployments to admin users only.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Secure Admissions Controller Webhooks**

  Implement secure methods to manage admissions controllers to prevent the injection of malicious code.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.

Elevation of privileges in the cluster

Attackers can gain more access than acquired during initial access.

Attackers might try to compromise Kubernetes clusters for various reasons, such as performing espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for cryptocurrency mining. They can elevate privileges on the cluster and potentially escape to the host through several methods:

- Exploiting and gaining access to a deployed Privileged container
- Deploying a Privileged container that they control or have configured
- Becoming cluster-admin
- Using or creating a hostPath mount
- Pivoting to a cloud environment for cloud-based deployments

Mitigation strategies include:

- **Strong Access Controls**

Implement strong access controls to limit who can deploy and manage resources within the cluster.

- **Restrict Privileged Containers**

  Limit the use of Privileged containers to admin users only. Use secure methods to deploy Privileged containers.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Restrict Access to Cloud Environments**

  Implement strong access controls and secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access to cloud environments.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.

Attacker avoids detection and hides activity on the cluster

An attacker can evade defenses or attempts to discover them or their activity.

Attackers might try to compromise Kubernetes clusters for various reasons, such as performing espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for cryptocurrency mining. They can hide their tracks through several methods:

- Clearing container logs
- Deleting Kubernetes events
- Using name-confusion with pods and containers of legitimate parts of the cluster
- Using a proxy server, VPN, TOR, or similar function to hide their source location

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can deploy and manage resources within the cluster.
- **Restrict Privileged Containers**

  Limit the use of Privileged containers to admin users only. Use secure methods to deploy Privileged containers.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Restrict Access to Cloud Environments**

  Implement strong access controls and secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access to cloud environments.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

Implement a secure way to store and manage kubeconfig files.

- **Logging and Monitoring**

  Implement logging and monitoring tools to detect and track malicious activity. Configure Kubernetes to retain event logs and container logs.
- **Name-Spacing and Tagging**

  Implement name-spacing and tagging for pods and containers to prevent name-confusion.
- **Network Security Groups**

  Configure network security groups to block proxy servers, VPNs, and TOR.

Acquisition of credentials to cluster resources

Attackers obtain credentials from applications, secrets, and configured identities.

Attackers might try to compromise Kubernetes clusters for various reasons, such as performing espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for cryptocurrency mining. They can obtain credentials to cluster resources through several methods:

- Listing Kubernetes secrets
- Mounting service principals
- Accessing container service accounts
- Reading configuration files for secrets
- Using the Identity Metadata Service to gain access to a token of a managed identity
- Installing a malicious admission controller webhook

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can access and manage resources within the cluster.
- **Restrict Privileged Containers**

  Limit the use of Privileged containers to admin users only. Use secure methods to deploy Privileged containers.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.
- **Secure Secrets Management**

  Implement secure secrets management practices to protect sensitive information.
- **Logging and Monitoring**

  Use tools like Falco or Sysdig to detect and prevent unauthorized activity.
- **Identity Metadata Service Security**

  Secure the Identity Metadata Service to prevent unauthorized access to tokens.
- **Admission Controller Security**

  Implement secure methods to manage admission controllers and regularly review and update them.

Pivot to other connected cluster resources

Attackers can use their existing foothold on the cluster to target other systems in the cluster or systems that are connected to the cluster.

Attackers might try to compromise Kubernetes clusters for various reasons, such as performing espionage, network reconnaissance, denial of service, data exfiltration and destruction, and hijacking resources for cryptocurrency mining. They discover new targets and perform lateral movement through several methods:

- **Discovery Methods**
  - Accessing the Kubernetes API Server
  - Accessing the Kubelet API
  - Performing network mapping from within the cluster
  - Accessing the Kubernetes Dashboard
  - Using the Instance Metadata API
- **Lateral Movement Methods**
  - Accessing cloud resources
  - Using their access to gain a token to use the container service account
  - Traversing the cluster network
  - Using credentials in configuration files to access other connected applications
  - Modifying files on the host through writable volume mounts
  - Accessing the Kubernetes Dashboard
  - Using Helm Tiller
  - Modifying the ConfigMap used by CoreDNS
  - Performing traditional ARP cache poisoning against cluster services

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can access and manage resources within the cluster.
- **Restrict Privileged Containers**

  Limit the use of Privileged containers to admin users only. Use secure methods to deploy Privileged containers.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.
- **Secure Secrets Management**

  Implement secure secrets management practices to protect sensitive information.
- **Logging and Monitoring**
- Use tools like Falco or Sysdig to detect and prevent unauthorized activity.
- **Instance Metadata API Security**

  Secure the Instance Metadata API to prevent unauthorized access to tokens.
- **Limit Lateral Movement**

  Implement measures to limit lateral movement within the cluster.

Container escape or breakout

Vulnerabilities in a container can allow an attacker to escape to the host.

An attacker can access an application vulnerable to Remote Code Execution (RCE) to run commands on the container's shell and perform actions on the host.

Attackers with shell access in the container can:

- Impact another container on the same host through container networking
- Perform operations on Kubernetes platform components or other cloud platform components
- Attack adjacent, vulnerable resources such as a network-based file system shared among containers, where they could read or modify data

Mitigation strategies include:

- **Strong Access Controls**

  Implement strong access controls to limit who can deploy and manage resources within the cluster.
- **Restrict Privileged Containers**

  Limit the use of Privileged containers to admin users only. Use secure methods to deploy Privileged containers.
- **Validate Container Images**

  Regularly update and patch container images in the registry. Use a vulnerability scanner to identify and address vulnerabilities in container images.
- **Secure HostPath Mounts**

  Use secure methods to manage hostPath mounts to prevent unauthorized access to the host system.
- **Patch Vulnerabilities**

  Regularly patch vulnerabilities to prevent exploitation.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms, such as OAuth or LDAP, to prevent unauthorized access.
- **Configure Insecure Interfaces Securely**

  Configure insecure interfaces securely and restrict access to the Kubernetes Dashboard.
- **Network Segmentation**

  Implement network segmentation to limit the exposure of insecure interfaces.
- **Protect the Kubeconfig File**

  Implement a secure way to store and manage kubeconfig files.
- **Secure Container Networking**

  Implement secure methods to manage container networking to prevent unauthorized access between containers.
- **Secure Kubernetes Platform Components**

  Ensure that Kubernetes platform components are securely configured to prevent unauthorized access.
- **Secure Cloud Platform Components**

  Implement security measures for cloud platform components to prevent unauthorized access.
- **Secure Network-Based File Systems**

  Ensure network-based file systems that are shared among containers are securely configured to prevent unauthorized access.

Exploiting software images compromised through the CI/CD process

Privileged or trusted users can inject malware or vulnerabilities into software binaries or containers during development to be exploited later.

An attacker might potentially be a malicious insider with privileged access to continuous integration and continuous delivery (CI/CD) platform components. The attacker can inject credentials, creating a potential backdoor, into Dockerfile templates or attempt to compromise containers during various stages of the CI/CD process. Each stage in which an infected image runs as part of a CI/CD process can be considered an attack surface, where vulnerable resources accessible by the infected image

can be compromised. Later, an attacker (who need not be the same malicious insider) might attempt to access an infected application run with privileges within a container, increasing the likelihood of other container-based attacks.

Mitigation strategies include:

- **Secure Access Controls**

  Implement strong access controls to limit who can access and manage CI/CD platform components.
- **Secure Authentication Mechanisms**

  Use secure authentication mechanisms, such as two-factor authentication, to prevent unauthorized access.
- **Monitoring for Suspicious Activity**

  Monitor CI/CD pipeline activities to detect potential security incidents.
- **Secure CI/CD Pipeline Practices**

  Regularly scan images for vulnerabilities, use secure authentication and authorization mechanisms, and monitor pipeline activities.
- **Container Security Measures**

  Use immutable images and regularly update container runtime environments.
- **Validate Docker Images and Containers**

  Ensure that all docker images and containers are validated and free from vulnerabilities.
- **Privilege Separation**

  Implement privilege separation for the CI/CD process to limit the impact of a compromised component.
- **Secure Storage and Management of Credentials**

  Use secure methods to store and manage user credentials.
- **Regular Updates and Patches**

  Regularly update and patch CI/CD platform software and container images.
- **Secure Access to CI/CD Process**

  Use secure methods to manage and control access to the CI/CD process.
- **Detection and Prevention of Container-Based Attacks**

  Implement tools and practices to detect and prevent container-based attacks.

By following these best practices, you can reduce the risk of successful CI/CD platform attacks and protect your AI Platform and its users.

Insecure configuration or misconfiguration

Insecure software component configurations can lead to security exploitation or sensitive information disclosure that attackers use during the reconnaissance phase.

Mitigation strategies include:

- **Secure Software Component Configuration Practices**

  Implement secure software component configuration practices to ensure that all components are properly configured.
- **Secure Coding Practices and Vulnerability Scanning**

  Use secure coding practices and vulnerability scanning to identify and remediate vulnerabilities.
- **Monitoring for Anomalies**

  Monitor software component configurations for anomalies and ensure that attackers have not tampered with them.
- **Regular Audits and Updates**

  Regularly audit and update software components to prevent configuration attacks.
- **Secure Defaults and Settings**

  Use secure defaults and settings to minimize the risk of exploitation.
- **Configuration Management and Hardening Techniques**

  Employ configuration management and hardening techniques to prevent security exploitation and sensitive information disclosure.
- **Component Validation and Testing**

  Use component validation, testing, and monitoring to prevent potential attacks.

- **Regular Security Updates and Patches**

  Apply regular security updates and patches to keep software components secure.
- **Access Control and Monitoring**

  Monitor and control access to sensitive information to prevent unauthorized access.
- **Vulnerability Scanning and Penetration Testing**

  Use techniques such as vulnerability scanning and penetration testing to identify and remediate insecure configurations.

By following these best practices, you can reduce the risk of successful software component configuration attacks and protect your AI Platform and its users.

Path traversal

An unauthorized attacker exploits a path traversal vulnerability to read or write from arbitrary files.

Path traversal vulnerabilities occur when software that determines filenames or paths to read/write data from/to is improperly validated or sanitized. This method allows attackers to manipulate file paths and access files outside the intended directory, potentially leading to unauthorized reading or writing of files.

Mitigation strategies include:

- **Input Validation and Sanitization**

  Implement proper input validation and sanitization for file paths and names to prevent malicious manipulation.
- **Secure APIs and Interfaces**

  Use secure APIs and interfaces for file access and manipulation to ensure safe handling of file paths.
- **Access Controls and Permissions**

  Configure access controls and permissions to restrict access to sensitive files and directories.
- **Regular Updates and Patching**

  Regularly update and patch software to address known vulnerabilities.
- **Monitoring and Alerting**

  Implement monitoring and alerting mechanisms to detect potential path traversal attacks.
- **Secure Storage Mechanisms**

  Use secure storage mechanisms for secrets, such as encrypted files or databases.
- **Web Application Firewall (WAF)**

  Consider using a WAF to detect and prevent path traversal attacks.
- **Secure Coding Practices**

  Follow secure coding practices to prevent path traversal vulnerabilities.
- **File System Permissions**

  Use file system permissions to restrict access to critical files and directories.
- **Log Review and Analysis**

  Regularly review and analyze log data to identify potential security threats.

Unauthorized access through certificate issues

Threat actors can perform spoofing or tampering attacks resulting from misconfigured certificates.

If not implemented correctly, certificate-based authentication such as TLS, Mutual TLS, HTTP over TLS (HTTPS), and VPNs can be subverted. Attackers can exploit misconfigured certificates to perform spoofing attacks or inject themselves into the communication path, leading to unauthorized access.

Threat actor behavior includes:

- Using false certificates to subvert authentication and gain unauthorized access.
- Performing Man-in-the-Middle (MiTM) attacks due to misconfigured certificates.

Mitigation strategies include:

- **Secure Certificate Management**

  Generate certificates from a trusted Certificate Authority (CA) and verify their authenticity before use.

- **Proper Configuration**

  Ensure proper configuration of certificate settings, including certificate lifetimes and renewal processes.
- **Certificate Pinning**

  Implement certificate pinning to ensure that the client only accepts a specific certificate or public key.
- **Mutual Authentication**

  Use mutual authentication to verify both the client and server identities.
- **Validation of Certificate Chains**

  Validate certificate chains to ensure that all certificates in the chain are trusted and have not been tampered with.

Malicious software installation

An attacker can corrupt or tamper with an installation package or patch to gain complete access to the customer environment. This type of attack can compromise the integrity and security of the system.

Mitigation strategies include:

- **Digital Signatures**

  Implement digital signatures for installation packages and patches. Verify the authenticity of updates before applying them to the system.
- **Software Update Verification**

  Properly verify software updates by checking digital signatures and ensuring updates are obtained from trusted sources.
- **Secure Boot and Firmware Validation**

  Implement secure boot and firmware validation to prevent tampering with the firmware.
- **Secure Communication Protocols**

  Use secure communication protocols such as HTTPS to encrypt communication between the PowerScale OneFS server and the client.
- **Regular Security Patches**

  Ensure that the PowerScale OneFS server has valid and up-to-date security patches.

General threats

Denial of service attacks

Denial of Service (DoS) threats aim to disrupt or degrade the availability or performance of any system or infrastructure platform, rendering it inaccessible to legitimate users. Attackers employ various techniques to overwhelm system resources, exhaust bandwidth, or exploit vulnerabilities to cause service disruptions. DoS attacks can result in financial losses, reputational damage, and customer dissatisfaction.

Mitigation strategies include:

- **Access Control and Rate Limiting**
  - Enforce proper access control and rate limiting to ensure that system resources are not exhausted.
  - Implement rate limiting on incoming traffic and system resources.
- **Monitoring and Updates**
  - Monitor the AI Platform for unusual traffic patterns.
  - Maintain up-to-date software versions to prevent exploitation of known vulnerabilities.
  - Review logs for anomalies.
- **Load Balancing and Traffic Distribution**

  Implement load balancing and content delivery networks (CDNs) to distribute traffic and reduce the burden on individual components.
- **Network Segmentation and Firewalls**
  - Segment the network using VLANs or firewalls to restrict access to AI Platform components.

- Isolate AI Platform segments by using separate VLANs.
- **Resource Monitoring and Intrusion Detection**
  - Implement resource monitoring tools to detect abnormal activity.
  - Enable intrusion detection systems for early threat identification.
- **Secure Network Protocols:**

  Configure secure network protocols such as TCP/IP.
- **Authentication and Authorization**
  - Implement robust authentication and authorization mechanisms for accessing the iDRAC BMC interface.
  - Enforce rate limiting on the management interface.
  - Implement IP blocking or whitelisting to restrict access from suspicious IP addresses.
- **Web Application Firewall (WAF)**
  - Use a web application firewall (WAF) to detect and prevent common DoS attacks.
  - Deploy a WAF module for each application deployed on your AI platform.
- **Logging and Monitoring**
  - Implement logging and monitoring tools that can detect and alert on anomalies.
  - Adjust security best practices and fine-tune the configuration of network rules to minimize unnecessary traffic.
- **Regular Software Updates:**

  Ensure the system's software is up to date with the latest security patches.

Data exfiltration

Data exfiltration refers to the unauthorized extraction or transfer of sensitive data from a system or network.

Attackers can exploit vulnerabilities, weak access controls, or employ various techniques to exfiltrate data. Common data exfiltration scenarios include File Transfer, Command and Control Channels, Steganography, Covert Channels, and Remote Access Trojans (RATs).

Mitigation strategies include:

- **Data Protection Measures**
  - Implement encryption, access controls, and secure data storage to protect sensitive data.
  - Use data loss prevention tools to detect and prevent sensitive data from being exfiltrated.
- **Monitoring and Scanning**
  - Regularly monitor for suspicious activity.
  - Perform vulnerability scanning and penetration testing to identify and remediate potential vulnerabilities.
  - Monitor network traffic for unusual patterns.
- **Access Controls**
  - Enforce robust access controls to prevent unauthorized access.
  - Implement secure communication protocols such as HTTPS and Mutual TLS.
- **Software and Plugin Updates**

  Regularly update software and plugins to ensure that any known vulnerabilities are patched.
- **Firewall Rules**
  - Enable firewall rules to block suspicious traffic.
  - Use compensating controls to protect authentication credentials that are sent in cleartext.
- **Trusted Networks**

  Ensure that the path between clients and the PowerScale OneFS server is on a trusted network.
- **Best Practices and Security Guides**

  Follow best practices for data-access protocols, FTP security, and HDFS security as outlined in the PowerScale OneFS Security Configuration Guide.

Unauthorized network traversal

Insufficient isolation and controls between network segments support attackers moving effectively unfettered to compromise other connected systems.

Inadequate segmentation refers to the improper isolation and lack of controls between different network segments within an environment. Network segmentation plays a critical role in maintaining the security and integrity of a system by preventing

unauthorized access and limiting the impact of potential security breaches. When network segmentation is not properly implemented or maintained, it can expose the environment to various risks and threats.

Common issues related to inadequate network segmentation include:

- Insufficient Firewall Configuration
- Weak Network Access Controls
- VLAN Misconfigurations
- Insufficient Pod-to-Service Isolation
- Insecure Pod Networking
- Inadequate Monitoring and Logging

Mitigation strategies include:

- **Network Segmentation**
  - Implement proper network segmentation using firewalls, access controls, and VLANs to isolate network segments.
  - Regularly review and update network segmentation to ensure that it remains effective and up-to-date.
- **Secure Communication Protocols**

  Use secure communication protocols, such as HTTPS, to protect data in transit.
- **Firewall Configuration**
  - Implement robust firewall configuration to control inbound and outbound traffic.
  - Enable host-based firewalls to control traffic at the host level.
- **Network Access Controls**
  - Strengthen network access controls to limit access to sensitive areas of the network.
  - Configure VLANs correctly to separate traffic between different network segments.
- **Pod and Service Isolation**

  Implement pod-to-service isolation and secure pod networking to prevent unauthorized access between pods and services.
- **Monitoring and Logging**
  - Ensure that monitoring and logging are adequate to detect and respond to potential security breaches.
  - Regularly monitor and log network traffic to detect unusual patterns or suspicious activity.
- **Best Practices and Security Guides**

  Follow best practices for network port usage, firewall default settings, and data-access protocols as outlined in the PowerScale OneFS Security Configuration Guide.

Unauthorized acquisition of user credentials

Credential theft is the unauthorized acquisition of user credentials, such as usernames and passwords, or 2FA codes that are sent over insecure channels, to gain unauthorized access to systems or sensitive information. Attackers employ various techniques such as phishing, keylogging, and password guessing to steal credentials. When obtained, these credentials can be abused to impersonate legitimate users, escalate privileges, or perform malicious activities in the system.

Mitigation strategies include:

- **Secure Password Management**
  - Implement secure password management practices.
  - Use multifactor authentication (MFA) to add an extra layer of security.
- **Authentication and Authorization**
  - Implement secure authentication and authorization mechanisms for the PowerScale OneFS server.
  - Use a secure way to store and manage user credentials.
  - Implement secure token-based authentication mechanisms.
- **Regular Updates and Patching**
  - Use a vulnerability scanner to identify and address vulnerabilities in the software.
- **Network Segmentation**
  - Implement network segmentation to limit exposure of insecure interfaces.
  - Use VLANs to separate traffic between different network segments.
- **Secure Communication Protocols**
  - Use secure communication protocols, such as HTTPS, to prevent 2FA codes from being intercepted.
  - Ensure that user credentials are not stored in plaintext.
- **Account Lockout Policies:**

    - Implement account lockout policies to prevent brute-force attacks.
- **User Education**

  Educate users about the risks of credential theft and how to protect themselves.
- **Monitoring and Logging**
    - Monitor system activity to detect and respond to potential security threats.
    - Regularly review and update security policies to ensure that they remain effective and up-to-date.

Subversion of the authentication mechanism

Authentication is a critical control for securing functionality and data, but improper authentication can be compromised.

Attackers can compromise authentication mechanisms by using several techniques, including credential stuffing and subverting 2FA code systems. Prevention of these threats is challenging, so detection and response are important.

Mitigation strategies include:

- **Robust Authentication Mechanisms**
    - Implement secure password policies, multifactor authentication (MFA), and secure password storage.
    - Use secure ways to generate, store, and manage 2FA codes.
- **Detection and Response**
    - Use detection and response tools to identify and prevent credential stuffing and 2FA code system subversion attacks.
    - Implement anomaly-based and machine learning-based detection to identify unusual authentication activity.
- **Secure Authentication and Authorization**
    - Implement secure authentication and authorization mechanisms for the PowerScale OneFS server.
    - Use a secure way to store and manage user credentials.
    - Implement secure token-based authentication mechanisms.
- **Regular Updates and Patching**
    - Regularly update and patch systems to ensure that they remain secure.
    - Use a vulnerability scanner to identify and address vulnerabilities in the software.
- **Network Segmentation**
    - Implement network segmentation to limit exposure of insecure interfaces.
    - Use VLANs to separate traffic between different network segments.
- **Secure Communication Protocols**
    - Use secure communication protocols, such as HTTPS, to prevent 2FA codes from being intercepted.
    - Ensure that user credentials are not stored in plaintext.
- **Account Lockout Policies**

  Implement account lockout policies to prevent brute-force attacks.
- **User Education**

  Educate users on the risks of credential stuffing and how to protect themselves.
- **Monitoring and Logging**
    - Monitor system activity to detect and respond to potential security threats.
    - Regularly review and update security policies to ensure that they remain effective and up-to-date.

Excessive privileges

Inadequate or ineffective access controls can result in unauthorized access to sensitive functionality, data, or resources within an application. This threat encompasses vulnerabilities and weaknesses in the application's access control mechanisms, which govern the granting or denying of permissions to users or entities. Attackers exploit these vulnerabilities to bypass access restrictions, elevate privileges, and perform actions beyond their authorized scope.

Mitigation strategies include:

- **Robust Access Control Mechanisms**
    - Implement secure authentication and authorization protocols.
    - Use role-based access control (RBAC) and attribute-based access control (ABAC) to manage permissions.
    - Apply the principle of least privilege to restrict user privileges.
- **Regular Reviews and Updates**
    - Regularly review and update access control policies to ensure that they are effective and aligned with business needs.

    - Regularly update and patch systems to ensure they remain secure.
- **Secure Authentication Mechanisms**
    - Ensure all users and entities are properly authenticated and authorized.
    - Use secure ways to store and manage user credentials.
- **Input Validation**
    - Implement input validation to prevent attackers from bypassing access controls through malicious input.
- **Monitoring and Logging**
    - Monitor system activity to detect and respond to potential security threats.
    - Implement secure logging to track access attempts and detect anomalies.
- **User Education**

  Educate users on the importance of access controls and how to use them effectively.
- **Network Segmentation**
    - Implement network segmentation to limit exposure of insecure interfaces.
    - Use VLANs to separate traffic between different network segments.
- **Detection and Response**
    - Use a vulnerability scanner to identify and address vulnerabilities in the software.
    - Implement a secure way to detect and prevent bypass of access control mechanisms.

Insecure data at rest

Insecure data at rest refers to the inadequate or weak protection of sensitive data in storage.

Cryptography is commonly used to secure sensitive information, such as passwords, payment details, or personal data, by encrypting it before storage. However, if cryptographic storage is implemented incorrectly or with weak algorithms, attackers might be able to recover or manipulate the stored data, leading to unauthorized access or compromise.

Mitigation strategies include:

- **Encryption and Key Management**
    - Configure encryption of secret data in etcd.
    - Use an external key management service or an encrypted secrets store provider.
    - Protect the secret passphrase with a strong and unique password, stored securely using a password manager or secrets management tool.
- **Secure Cryptographic Storage**
    - Ensure that the Ansible Vault is properly configured to use a secure encryption algorithm and a strong key.
    - Use strong encryption algorithms and implement secure key management.
- **Proper Configuration**
    - Ensure proper configuration of cryptographic storage.
    - Implement proper encryption and use secure storage solutions.
- **Regular Updates and Patching**
    - Regularly update and patch systems to ensure that they remain secure.
    - Perform the OneFS rekey operation regularly.
- **Monitoring and Auditing**
    - Monitor data at rest for anomalies and ensure it is not tampered with.
    - Regularly audit and update data protection mechanisms to prevent data at rest attacks.
- **User Education**

  Educate users on the importance of data protection and how to use encryption effectively.

Insufficient logging

Insufficient logging means that there are inadequate mechanisms to track and detect security incidents.

Insufficient logging and monitoring practices make it difficult to identify and respond to security incidents in a timely manner. This threat encompasses various issues that are related to logging, monitoring, and incident detection, such as lack of centralized logging, limited event correlation, and insufficient alerting mechanisms.

Mitigation strategies include:

- **Centralized Logging Solution**

    - Implement a centralized logging solution, such as Fluentd, Fluent Bit, or ELK Stack, to collect and analyze logs from Kubernetes components.
    - Configure Omnia to create more detailed and centralized logs.
- **Event Correlation and Alerting**
    - Configure event correlation and alerting mechanisms to detect security incidents.
    - Use tools that enable event correlation and set up alerting mechanisms for critical events.
- **Third-party Monitoring Solutions**
    - Use third-party solutions, like Prometheus and Grafana, for monitoring and visualization.
    - Consider using a Security Information and Event Management (SIEM) system to collect and analyze logs from various sources.
- **Logging and Monitoring Best Practices**
    - Implement logging and monitoring best practices.
    - Regularly review and analyze log data to identify potential security threats.
- **Incident Response Plan**

  Set up an incident response plan to respond to detected security incidents promptly.
- **Regular Updates and Patching**

  Apply regular security updates and patches to ensure that the logging and monitoring systems remain secure.

Insecure logging

Attackers can manipulate log contents to hide activities or impact other systems.

Insecure logging can allow attackers to tamper with the integrity of audit logs to mislead or hide other elevated attacks. Attackers can exploit vulnerabilities in software to perform log injection, log forging, and log poisoning. They can also cause a denial of service to the target system or downstream consumers of logs through vulnerabilities in log parsers.

Mitigation strategies include:

- **Tamper-Evident and Tamper-Proof Logging**
    - Implement a logging solution that provides tamper-evident and tamper-proof logging capabilities.
    - Use secure protocols for logging, such as TLS or SSH.
- **Log Parser Security**
    - Configure log parsers to detect and prevent log injection, log forging, and log poisoning.
    - Secure log parsers and implement input validation to prevent these attacks.
- **Regular Updates and Patching**
    - Regularly update and patch systems to ensure that they remain secure.
    - Monitor log parser vulnerabilities and implement rate limiting on log messages.
- **Monitoring and Alerting**
    - Implement monitoring and alerting mechanisms to detect potential log tampering or denial of service attacks.
    - Monitor logs for signs of unauthorized access or manipulation.
- **Secure Logging Mechanisms**
    - Configure Omnia to use secure logging mechanisms, such as encryption and digital signatures.
    - Implement logging best practices, such as regular log rotation and archiving.
- **Log Integrity Checking**
    - Implement digital signatures for audit logs and use secure logging protocols.
    - Configure logging to remote syslog servers and use log encryption.
- **Audit Log Encryption**
    - Implement audit log encryption and use secure audit log storage.
    - Regularly review and analyze log data to identify potential security threats.

Hardware threats

Introduction of internal implants

There can be a risk of an attacker injecting persistent malicious firmware or attaching a malicious device into platform hardware.

Look for opportunities where unauthorized devices can be installed on ports that reside inside the chassis. Attacks can occur by opening the chassis, inserting hardware or malware into the platform, and then closing the chassis, or by inserting a tampered field-replaceable unit (FRU) during system maintenance. This method can conceal a persistent implant, allowing adversaries to continuously access the platform a significant time after the initial intrusion event. Implants can vary, taking the form of malware code injected into the BIOS, EC, MEFW, or other microcontrollers with privileged access to other buses and data.

Physical manifestations can include wireless devices for data exfiltration, remote control, keyloggers, video capture, audio capture, DMA engines, control devices for JTAG or UART interfaces, or bus sniffing and traffic altering tools for man-in-the-middle attacks.

Mitigation strategies include:

- **Physical Security Measures:**
  - Implement secure storage and handling of the product.
  - Secure ports inside the chassis to prevent unauthorized devices from being installed.
- **Firmware and Software Updates:**
  - Use secure firmware and software updates.
  - Regularly review and update the firmware and software of the PowerEdge servers.
- **Intrusion Detection and Prevention:**
  - Implement intrusion detection and prevention systems to detect and respond to potential attacks.
  - Monitor system activity to detect and respond to potential security threats.
- **Secure System Maintenance:**
  - Implement secure system maintenance procedures to prevent tampered FRUs from being inserted during system maintenance.
  - Ensure all devices that are installed inside the chassis are properly authorized and validated.
- **Secure Hardware Components:**
  - Use secure hardware components with integrated security features to prevent malware code injection.
  - Implement secure firmware updates to prevent malware code injection.
- **Regular Review and Update:**

  Regularly review and update security policies to ensure that they remain effective and up-to-date.

Unauthorized changes using a service interface

Weak or inadequate protection on physical Service Access interfaces might lead to unauthorized changes.

Unauthorized changes to system configuration data or firmware through service access interfaces might lead to degradation of security features such as secure boot, signature checks, and installation of firmware or hardware components.

Mitigation strategies include:

- **Authentication and Authorization:**

  Ensure proper implementation of authentication and authorization for access to service functionality.
- **Protection of Service Interfaces:**
  - Verify protections for access to service interfaces.
  - Detect and log any accesses to chassis or case opening and service ports.
- **Disabling Service Functionality:**

  Disable service functionality after each use.
- **Documentation and Logging:**
  - Keep records of services, accounts, and logs.
  - Implement logging best practices, such as regular log rotation and archiving.
- **Monitoring and Tools:**
  - Install monitoring tools such as Prometheus and Grafana.
  - Implement monitoring and alerting mechanisms to detect potential security incidents and anomalies.
- **Configuration Management:**
  - Use configuration management tools like Omnia to ensure proper implementation of security measures.
  - Document service interfaces, accounts, and log related information.

Use of hardware fault injection to compromise a device

Inadequate protection from voltage and clock defects or other fault injection attacks can affect the operation of a device.

With physical access to a product, it is possible to perform fault injection on power, signal, or clock pins to cause a circuit to alter its behavior, such as skipping or changing an instruction. Injection of induced voltage from nearby electromagnetic pulses might also cause similar effects. These techniques can be used to bypass security features such as authentication or protections on debug or privileged functionality.

Mitigation strategies include:

- **Physical Security Measures:**
  - Implement secure storage and handling of the product.
  - Implement secure physical access controls, such as locking mechanisms and access controls.
- **Fault-Tolerant Design Techniques:**
  - Implement secure boot mechanisms to prevent tampering.
  - Use secure firmware updates to prevent tampering.
- **Secure Authentication and Authorization:**
  - Implement secure authentication and authorization mechanisms.
  - Manage debug and privileged functionality securely.
- **Network Segmentation:**
  - Implement network segmentation to limit exposure of insecure interfaces.
  - Use secure protocols for data transfer and communication.
- **Fault Injection Detection:**
  - Implement fault injection detection mechanisms to detect and respond to fault injection attacks.
  - Monitor system activity to detect and respond to potential security threats.
- **Regular Review and Update:**

  Regularly review and update security policies to ensure that they remain effective and up-to-date.

Weak or inadequate protection of hardware debug interfaces

Unauthorized access to hardware debug interfaces can expose systems to various security risks, including malware infections, unauthorized access, and data breaches.

Mitigation strategies include:

- **Secure Hardware Debug Interface Management:**
  - Implement secure hardware debug interface management to prevent hardware debug interface attacks.
  - Use secure boot and secure firmware to prevent malware infections.
  - Monitor hardware debug interfaces for anomalies and ensure that they are not tampered with.
- **Regular Audits and Updates:**
  - Regularly audit and update the hardware debug interfaces to prevent hardware debug interface attacks.
  - Use access controls and authentication to prevent unauthorized access.
- **Access Controls and Authentication:**
  - Implement secure access controls, use authentication, and authorization mechanisms.
  - Employ network segmentation and access controls to prevent unauthorized access and data breaches.
- **Secure Boot Mechanisms:**
  - Implement secure boot mechanisms to prevent tampering.
  - Use hardware-based security features such as Trusted Platform Modules (TPMs).
- **Restrict Access to Debug Interfaces:**
  - Restrict access to debug interfaces.
  - Use techniques such as firmware validation and secure firmware updates to prevent unauthorized access.

Conclusion

Summary

This white paper is a guide to security best practices for generative AI in the enterprise, with application to the Dell AI Platform designs and to AI systems in general.

It paper describes many of the potential threats to AI systems and offers guidance on how to mitigate them. It describes some common security configuration procedures, followed by guidance related to categories of potential threats and their respective mitigations. It presents a wide range of considerations, ranging from general hardware- and software-based threats that could apply to any IT system, to threats that are specific to AI platforms.

While this white paper is relatively comprehensive, the recommendations presented here do not provide a complete security evaluation of the Dell AI Platforms in their entirety or their individual components, nor a complete threat and risk analysis specific to individual use cases. For more guidance, go to the Dell Security and Trust Center or Dell AI Professional Services.

We value your feedback

Dell Technologies and the authors of this document welcome your feedback on this document and the information that it contains. Contact the Dell Technologies Solutions team by email.

References

Dell Technologies security resources

The following additional security resources are available from Dell Technologies.

- Dell Technologies Security and Trust Center
- Dell PowerEdge Server BIOS Security Configuration Guide
- PowerScale OneFS Security Configuration Guide
- Enterprise SONIC Distribution by Dell Technologies Security Configuration Guide
- Omnia Security Configuration Guide
- Dell Automation Platform Security Configuration Guide
- Dell AI Professional Services

Dell AI Platform design guides

The security practices described in this document are intended to be implemented in conjunction with the following Dell AI Platforms, all part of the Dell AI Factory portfolio.

- Dell AI Platform with AMD
- Dell AI Platform with Intel
- Dell AI Platform with NVIDIA including NVIDIA GPUs, Dell Networking, and Open-source Software Stack
- Dell AI Platform with NVIDIA including NVIDIA GPUs, Dell Networking, and NVIDIA Software Stack
- Dell AI Platform with NVIDIA including NVIDIA GPUs, NVIDIA Networking, and Open-source Software Stack

Other resources

The following security resources are available for the respective technologies.

- xCAT Security Configuration
- NSA Kubernetes Hardening Guide
- Kubernetes Security